



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

Reinforcement Learning

Project 3: Temporal Difference Learning

Mark D. Happel, D.Sc.

Chief Engineer, Unmanned Surface Vessel Combat System

Information Fusion and Artificial Intelligence Group

Johns Hopkins University Applied Physics Laboratory

mark.happel@jhuapl.edu

240-228-9848 (office)

Project 3 assignment (1)

- Task:
 - You are to implement (1) an on-policy SARSA reinforcement learning agent and (2) an off-policy Q-learning reinforcement learning agent that can learn to play the puzzle described on the following slides. These agents should consist of Python classes and any necessary accompanying functions.
- Given:
 - One Python files - an 'environment' file – provided by me
 - The details of the environment will be provided in the following slides.

Project 3 assignment (2)

- Procedure:
 1. In a file named “[Project3_agentS.py](#)”, implement the class “[SARSA_0](#)”.
 2. In a second file named “[Project3_agentQ.py](#)”, implement the class “[QAgent](#)”.
 3. Test the member functions of your classes using test functions in separate files
 4. Implement a “[Project3_main.py](#)” file with which to run the agents and environment as described below.
 5. When you are satisfied that your implementation is correct, combine the files and run the experiment described in the following slides.
 6. Collect the data required (as a minimum) and document it in a technical paper.
 7. Submit the technical paper and your source code.

Puzzle: “Gold Explorer”

	0	1	2	3	4	5	6	7
0	 0	 1	2	3	4	5	6	7
1	8	 9	10	11	12	 13	14	 15
2	16	 17	18	19	20	21	22	23
3	24	25	26	27	 28	29	30	31
4	 32	33	34	35	36	37	38	39
5	40	41	42	43	44	45	46	47
6	48	 49	 50	 51	52	53	54	55
7	56	57	58	59	60	61	 62	63



Tokens

	The Explorer. He moves around the grid looking for gold.
	Empty space. $R = -1$
	Mountain. This grid space cannot be entered from any direction.
	Mine field. If he enters, he will lose. Terminal state, $R = -30$.
	Gold coins! $R = +15$. Not a terminal state, but he can only obtain the gold once.
	The motherlode! If he enters, he wins. Terminal state, $R = +30$.

The Explorer



- The explorer's goal is to find the shortest route to obtain the most gold
- Always starts in state 0
- Can move north (N), south (S), east (E), or west (W)
- Each transition into a non-terminal space receives a penalty of -1
- If he tries to cross the outer border of the grid world or tries to enter a mountain space, he returns to the same space and receives a penalty of -1

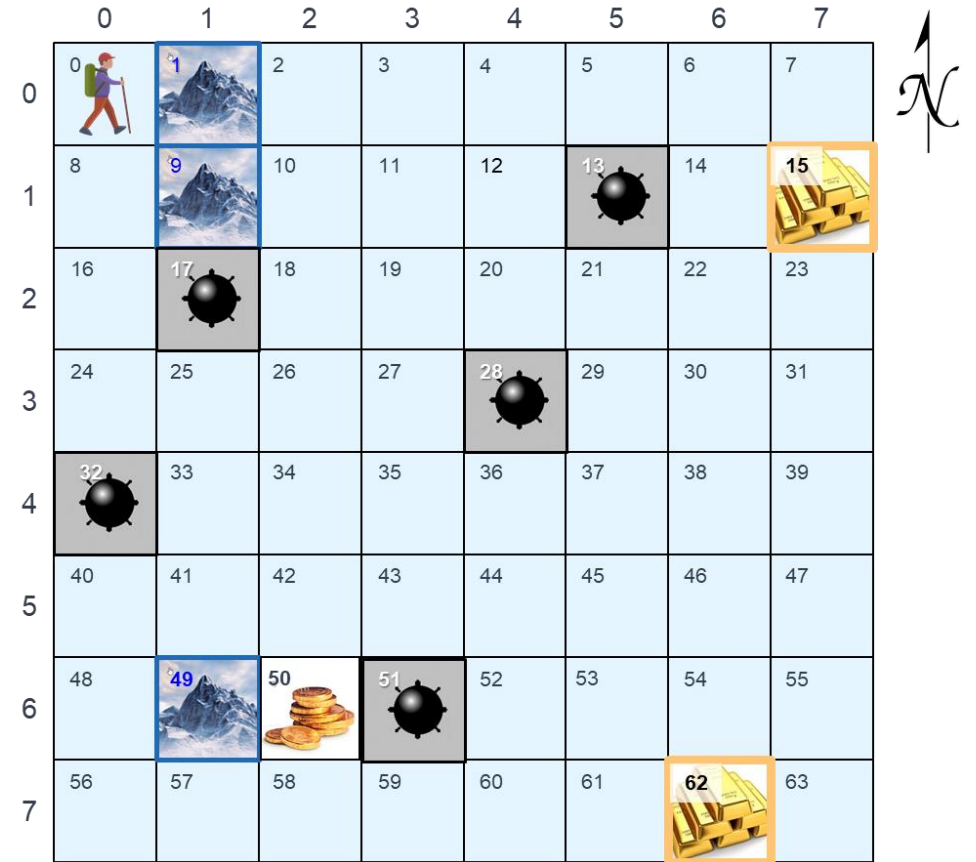


Project 3: Environment

- States

- The state transition function is deterministic
- The 'game board' has 64 grid locations (0 to 63):
 - 54 decision squares
 - 7 terminal squares
 - 3 squares that are undefined / cannot be entered

- **Oops – this won't work!**

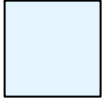
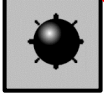
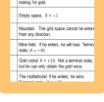


The coins require an additional feature

	0	1	2	3	4	5	6	7
0								
1	8							
2	16							
3	24	25						
4		33						
5	40	41	42	43	44	45	46	47
6	48				52	53	54	55
7	56	57	58	59	60	61		63

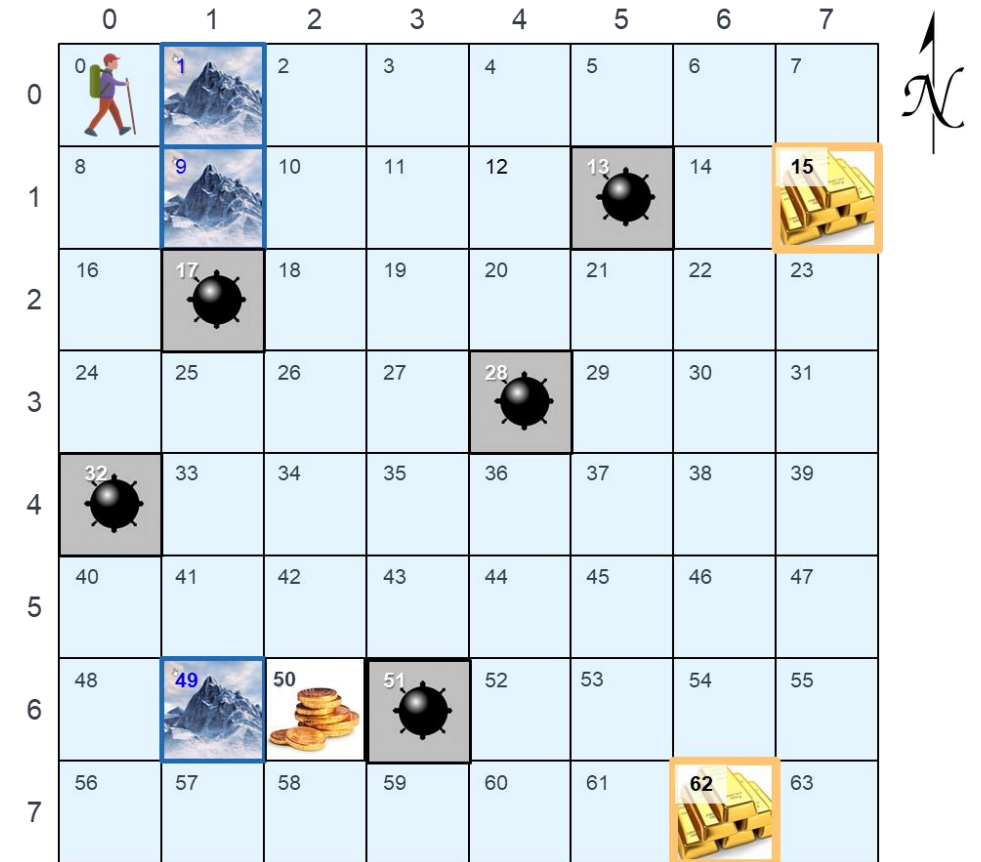
We want the agent to learn to take the coins when they are available. However, by using only the grid location as the state definition, the agent can't know whether the coins are available or not. The agent will learn to go to grid #50, and then will learn not to go to grid #50.

Tokens

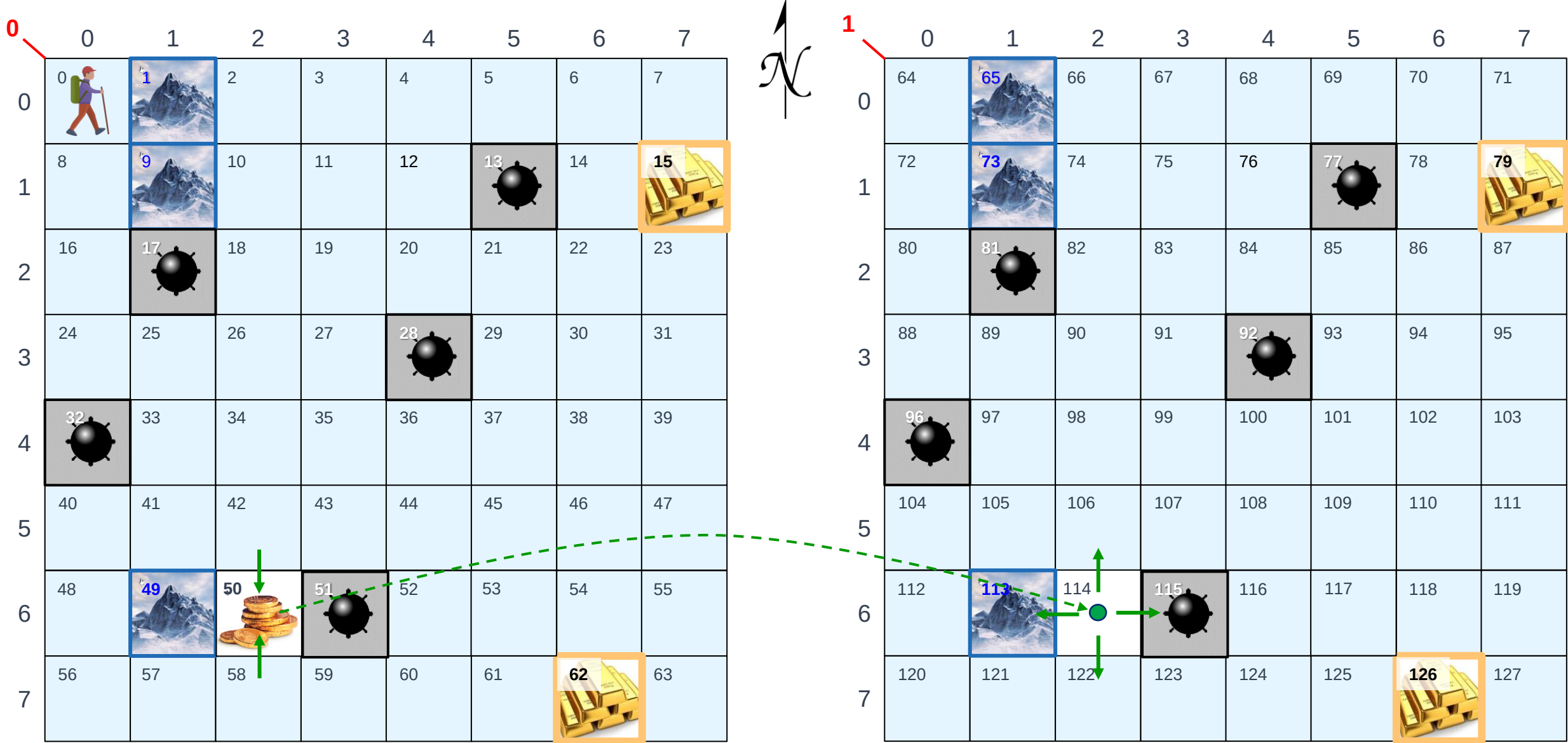
	The Explorer. He moves around the grid looking for gold.
	Empty space. $R = -1$
	Mountain. This grid space cannot be entered from any direction.
	Mine field. If he enters, he will lose. Terminal state, $R = -30$.
	Gold coins! $R = +15$. Not a terminal state, but he can only obtain the gold once.
	The motherlode! If he enters, he wins. Terminal state, $R = +30$.

Project 3: Environment

- Square 50 needs special treatment
 - The first time square 50 is entered during an episode, the explorer is awarded gold (signified by a reward)
 - All other times during the same episode, the explorer receives nothing
 - If only the grid location is used as the state definition, the problem will not be Markovian
- A good way to handle this will be to introduce another feature to the state definition
 - Add another feature to indicate whether the gold in square 50 has been obtained
 - Now a transition into square 50 causes a further transition to square 114



Modifying the state definition



Project 3: Environment

- Using the modified state definition
 - There are 128 states (0 to 127):
 - 107 decision states
 - 14 terminal states (only two are unique)
 - 1 state (#50) that automatically transitions to another state (#114)
 - 6 states that are undefined / cannot be entered (these do not need to be represented)
 - Environment will send the agent an integer s , $0 \leq s \leq 127$, representing the state
 - Internally, the environment will represent the state as an octal number, <gold bit><row><column>
 - The state transition function is deterministic
- Rewards
 - The rewards are specified on slide 4
 - The reward function is deterministic
- Actions
 - The environment expects to receive an integer representing one of the 4 possible actions, coded as:
 - 'N' = 0, 'E' = 1, 'S' = 2, 'W' = 3

The environment class *GoldExplorer()* (1)

- Interface functions:
 - `get_number_of_states()`
 - Returns the (constant) integer number of states defined by the environment.
 - `get_number_of_actions()`
 - Returns the (constant) integer number of actions allowed by the environment.
 - `get_terminal_states()`
 - Returns a set containing the state numbers of the terminal and undefined states
 - This will permit the agent to distinguish between terminal states and decision states when setting optimistic values
 - `get_state()`
 - Returns the current state of the environment (an integer value).
 - `reset(exp_starts)`
 - Resets the environment to the beginning of the episode. If *exp_starts* is true, then a random initial state is chosen. Otherwise, the initial state will be S_0 . Returns the initial state.

The environment class *GoldExplorer()* (2)

- Interface functions (continued):
 - `execute_action(action)`
 - executes the action given by *action*.
 - causes the next state to be determined, the state of the environment to be updated, and the applicable reward to be calculated.
 - returns the new state, the reward, and the terminal flag (in that order)
 - Note:
 - States and actions are represented by integers ≥ 0 .
 - Rewards are represented by integers R ; $-30 \leq R \leq 30$.
 - The terminal flag is {True, False}

Methods: Data collection & processing (1)

- a. Start with the untrained agent $i = 1$
- b. Train the agent until performance increases begin to asymptote
 - During training, collect the following data for each episode:
 - $G_n(i)$, the return for agent i from episode n
- c. Repeat steps a & b for the remaining agents $i = 2$ to 50
- d. Calculate $\bar{G}_n$, the mean return from episode n

$$\bar{G}_n = \frac{1}{50} \sum_{i=1}^{50} G_n(i)$$

- Also calculate the standard error of the mean for each episode
- e. Calculate $\bar{G}$, the mean return for the ‘trained’ learning agents by averaging the $\bar{G}_n$ values for the last 50 episodes

Experiment to be performed

- Start with the following hyperparameters:
 - $\alpha = 0.1$
 - $\gamma = 0.95$
 - Use ε -greedy with $\varepsilon = 1.0$ and decrease it over time to $\varepsilon = 0.01$
 - Use the 'decayed-epsilon-greedy' method (same as Project #2)
 - Choose an N such that the small ε is set around the time that the performance begins to asymptote
- Run both the SARSA agent and the Q-Learning agent on the puzzle.
- Try exploring starts for both SARSA and Q-Learning
- Compare the performance metrics that you have obtained for the SARSA agent and the Q-learning agent

Technical paper content

- Title page
 - Project 3
 - Name, Class/Section
 - Date
- Methods Section
 - Describe in detail what you actually did
 - Only include what worked
 - Write it as though everything you did worked the first time
- Results Section
 - Describe your results using text, charts, tables, etc. Include explanations about any inferences or conclusions you drew from your results
- Discussion Section
 - A short section describing the broader implications from your results and recommendations for future studies
- References (if appropriate)

Project 3 submission

- Submission
 - Submit as a minimum the [Project3_agentS.py](#), the [Project3_agentQ.py](#), and the [Project3_main.py](#) source code files and the documented results via Canvas
 - do not 'zip' or otherwise archive these files – submit them as separate files
 - do not submit a Jupyter notebook – if you use one for development, generate a source code file from your notebook script
 - if you don't know how to do this, check out <https://mljar.com/blog/convert-jupyter-notebook-python/>
 - You do not need to submit the environment file that was provided for you
 - You do not need to submit the code you wrote to test your environment
 - You do not need to submit your raw data



JOHNS HOPKINS
APPLIED PHYSICS LABORATORY